

REVISED — SENIOR APPROVED
SCOPE

Doctor Prescription Audio Recorder App

Revised Technical Architecture — Post Senior Review

Date: 21 April 2026 | Platform: Android (Flutter) | Version: 2.0

Project	Audio-based prescription recording system for doctors
Platform	Android (Flutter)
Database	FTP Server — stores all recordings, doctor profiles, login data
Auth	FTP-based login with per-doctor folder isolation
ML feature	Basic — silence/noise detection for clean recordings
Removed	Whisper API, LLM structuring, send/share options
Phases	4 focused development phases

1. What Changed After Senior Review

Feature	Previous plan	Revised plan
Transcription	OpenAI Whisper API	Removed
AI structuring	Claude / GPT-4 LLM	Removed
Send options	WhatsApp, PDF, Firebase push	Removed
Database	Firebase Auth + Firestore	FTP server for everything
Doctor login	Firebase Auth	FTP credentials per doctor
ML	Not planned	Basic noise/silence detection
Recording	Record + pause + resume	Same + auto-pause on calls
My recordings	Upload / download / delete	Same — via FTP

2. Revised Application Scope

The application now has a clean, focused set of screens and features approved by the senior team:

Screen	Features
Login screen	Doctor enters FTP credentials (username + password). App authenticates against FTP server. Session saved locally.
Recording screen	Start recording, pause manually, resume. Auto-pause when call comes in or audio focus lost. Resume banner shown after call. Timer and waveform displayed.
My recordings	List of all recordings in doctor's FTP folder. Upload new recording, download existing, delete. Each doctor sees only their own files.
Settings / logout	View doctor profile info. Logout clears session and returns to login screen.

3. System Architecture

The entire backend is the FTP server. There is no Firebase, no external API, no cloud database. The FTP server holds everything: doctor login credentials, profiles, and audio recordings.

Layer 1 — Flutter app (doctor's device)	
Login screen	FTP auth — validates credentials directly against FTP server
Recording screen	record package, phone_state for call detection, audio_session for focus
My recordings	ftpconnect package — list, upload, download, delete files
ML module	On-device: silence detection, basic noise filtering on audio file
Local cache	hive — stores session token, recording metadata, offline state

Layer 2 — FTP server (single backend)	
Doctor auth	/users/{username}.json — stores hashed password and profile info
Recordings folder	/recordings/{username}/ — private folder, one per doctor
Profile data	Name, specialisation, hospital stored as JSON in doctor folder
Access control	FTP user accounts — each doctor can only access their own directory

FTP Server Folder Structure

Path on FTP server	Contents
/users/dr_mehta/	profile.json (name, hospital, specialisation)
/users/dr_mehta/recordings/	rec_20260421_001.m4a, rec_20260421_002.m4a ...
/users/dr_sharma/	profile.json
/users/dr_sharma/recordings/	dr_sharma's audio files only

4. Recording & Interrupt Flow

The recording screen handles three types of interruptions automatically: incoming phone calls, outgoing calls, and any other event that causes the system to take away audio focus (e.g. navigation apps, alarms). The flow is handled by the `phone_state` and `audio_session` packages.

Step	Trigger	App behaviour
	1 Doctor taps Record	<code>recorder.start()</code> — audio saved to local path, timer starts, waveform shown
	2 Incoming / outgoing call	<code>phone_state</code> fires event — <code>recorder.pause()</code> called automatically
	3 During call	Recording stays paused, timestamp preserved, UI shows paused banner
	4 Call ends	<code>phone_state</code> fires IDLE — banner: 'Call ended. Tap to resume recording'
	5 Doctor taps Resume	<code>recorder.resume()</code> — continues from exact pause point
	6 Doctor taps Stop	<code>recorder.stop()</code> — file saved locally, upload to FTP initiated

5. Basic Machine Learning Feature

As approved by the senior team, a lightweight on-device ML feature will be included. This runs entirely on the device — no external API needed. The feature is silence detection and basic noise filtering applied to the recorded audio file before it is uploaded to the FTP server.

ML feature	What it does	Library
Silence trimming	Detects and removes silent gaps at start/end of recording	<code>flutter_sound</code> or noise package
Noise gate	Reduces background hiss and low-level ambient noise	On-device audio processing
Speech detection	Flags segments where no speech was detected (very quiet parts)	Amplitude threshold analysis

All ML processing happens locally after the doctor stops recording and before the file is uploaded. The doctor sees a brief 'Processing audio...' indicator. The cleaned file is then stored both locally and uploaded to their FTP folder.

6. Technology Stack

Package	Purpose
record: ^5.1.2	Core audio recording — M4A output
audio_waveforms: ^1.0.5	Live waveform display during recording
just_audio: ^0.9.36	Playback of saved recordings in My Recordings
phone_state	Detect incoming/outgoing calls for auto-pause
audio_session	Handle system audio focus loss (alarms etc.)
ftpconnect	FTP upload, download, delete, list files
path_provider: ^2.1.2	Access local device storage paths
hive: ^2.2.3	Local cache — session, metadata, offline state
hive_flutter: ^1.1.0	Flutter adapter for Hive
permission_handler	Request mic + storage permissions
flutter_riverpod	State management across screens
noise / flutter_sound	On-device audio processing for basic ML

7. Flutter Project Folder Structure

lib/main.dart	App entry, Hive init, FTP service init
lib/core/ftp/ftp_service.dart	All FTP operations — login, upload, download, delete, list
lib/core/auth/auth_controller.dart	Login/logout logic, session persistence in Hive
lib/core/ml/audio_processor.dart	Silence trimming + noise gate on recorded file
lib/features/login/	Login screen UI
lib/features/recording/	Recording screen, waveform, timer, interrupt handler
lib/features/my_recordings/	Recording list, FTP file browser, download/delete
lib/features/settings/	Doctor profile view, logout button
lib/shared/models/	Doctor model, Recording model
lib/shared/widgets/	Reusable UI components

8. Development Phases

Phase	Focus	Deliverable	Time
1	FTP auth + login screen Connect to FTP, validate credentials, save session, logout	Working login/logout with FTP	Week 1
2	Recording + interrupts Record, pause, resume, auto-pause on calls, save locally	Full recording screen	Week 1-2
3	My Recordings + FTP sync List files, upload, download, delete via FTP	My Recordings screen	Week 2-3
4	ML audio processing Silence trimming, noise gate, integrate into upload flow	Clean audio on FTP upload	Week 3-4

9. Approval Sign-off

This revised proposal incorporates all feedback from the senior review. The scope is focused, technically sound, and achievable within the proposed timeline. Please sign below to approve commencement of development.

Role	Name	Signature	Date
Senior / Project Lead			
Technical Reviewer			
Developer			

Revised per senior feedback. Scope is final. Development begins upon signature.